
Laboratoire 06

Portage de Linux sur la carte DE-1

Départements : TIC
Unité d'enseignement SCF

Auteur : Urs Behrmann
Professeurs : Alberto Dassatti
Yann Thomas
Assistant : Clément Dieperink
Classe : SCF
Salle de labo : A07
Date : 21.04.2026

Table des matières

1	Introduction	3
2	Flash de la carte SD	3
3	Vérification du démarrage	3
4	Compilation du noyau Linux 6.18	3
5	Création du device tree DE1	4
6	Génération de l'image ITB	4
7	Copie sur la carte SD et test	4
8	Test du bridge lwhps2fpga et devmem	5
9	Adaptation du programme du laboratoire 5 sous Linux	5
10	Reprogrammation du FPGA et problème de licence	6
11	Utilisation d'outil IA	7

1 Introduction

Ce rapport décrit les étapes suivies pour démarrer Linux sur la carte DE1-SoC, mettre à jour le noyau vers la version 6.18 avec un device tree adapté, puis porter le programme de test du laboratoire 5 en application *user-space* sous Linux (`/dev/mem`, `scp`, tests sur la carte).

2 Flash de la carte SD

J'ai utilisé le script fourni pour écrire l'image sur la carte SD : `./deploy_img.sh -d sda -t de1soc -i ./de1soc.img`

Avant d'exécuter le script, j'ai vérifié le périphérique avec `lsblk` pour ne pas écraser le disque du PC. La carte apparaissait comme `/dev/sda` (14.6G, amovible).

3 Vérification du démarrage

J'ai vérifié le boot sur la console série avec : `picocom -b 115200 /dev/ttyUSB0`

Le démarrage était correct jusqu'au login. Ensuite j'ai configuré le réseau du PC dans le même sous-réseau que la carte (192.168.0.X/24) pour accéder en SSH avec `ssh root@192.168.0.2`.

4 Compilation du noyau Linux 6.18

J'ai récupéré les sources Linux mainline avec une profondeur limitée : `git clone -depth 1 -branch v6.18 https://github.com/torvalds/linux.git`

La toolchain attendue dans `/opt` n'était pas présente sur mon système Kubuntu natif, donc j'ai utilisé celle déjà installée : `export ARCH=arm et export CROSS_COMPILE=arm-linux-gnueabi`

Ensuite j'ai compilé avec la configuration SoCFPGA : `make socfpga_defconfig make -j$(nproc) zImage dtbs modules`

5 Création du device tree DE1

Dans Linux 6.18, les DTS SoCFPGA sont dans : `arch/arm/boot/dts/intel/socfpga/`

J'ai copié `socfpga_cyclone5_de0_nano_soc.dts` vers un nouveau fichier pour DE1, puis j'ai fait les modifications demandées :

- suppression des nœuds `gpio0`, `gpio1`, `gpio2`
- suppression du nœud `i2c0`
- modification des bootargs à `root=/dev/mmcblk0p3 rw rootwait` avant `earlyprintk`

J'ai ensuite ajouté le nouveau `.dtb` dans le `Makefile` du dossier SoCFPGA et recompilé le device tree.

6 Génération de l'image ITB

J'ai modifié `de1soc.its` pour inclure les bons fichiers (`zImage` compilé et le nouveau `.dtb`), puis j'ai généré l'image : `mkimage -f de1soc.its de1soc.itb`

Dans mon fichier `de1soc.its`, les chemins sont des chemins absolus vers l'emplacement local où j'ai récupéré Linux 6.18.0, c'est-à-dire dans mon dossier personnel (`~/linux-6.18.0`).

La commande `mkimage -l de1soc.itb` a confirmé que l'image contenait bien le kernel et le device tree.

7 Copie sur la carte SD et test

J'ai monté la partition BOOT de la carte SD (`/dev/sda1`) et copié `de1soc.itb`. Ensuite j'ai démonté la carte proprement.

Après redémarrage de la carte, la validation finale s'est faite via la console série et la commande `uname -r` pour vérifier la version du noyau.

8 Test du bridge lwhps2fpga et devmem

J'ai utilisé `devmem` sur la carte pour lire la mémoire physique du bridge. La lecture à l'adresse de base `0xFF200000` (mot 32 bits, option `w`) renvoie la constante attendue de l'IP AXI (`0xBADB100D`), ce qui confirme que l'accès au bridge fonctionne.

Pour les offsets du design de base (switchs `0x4B000`, touches `0x4B004`), la commande `devmem` de BusyBox a parfois renvoyé une erreur `mmap: Invalid argument` sur certaines adresses, alors qu'un accès via `/dev/mem` dans un programme (voir section suivante) fonctionnait. J'ai donc gardé `devmem` pour la lecture à `0xFF200000` et le programme pour les autres offsets.

9 Adaptation du programme du laboratoire 5 sous Linux

J'ai repris la logique du laboratoire 5 (registres AXI, LEDs, afficheurs, touches) et je l'ai portée en application *user-space* sous Linux :

- ouverture de `/dev/mem` et `mmap` sur une fenêtre assez grande à partir de `0xFF200000` pour couvrir à la fois les registres de l'IP (petits offsets) et les emplacements du bridge utilisés au laboratoire 6 (offsets `0x4B000` / `0x4B004`) ;
- remplacement des accès directs à l'adresse physique par des lectures / écritures via le pointeur retourné par `mmap` ;
- compilation croisée avec `arm-linux-gnueabi-gcc` et un `Makefile` dans `labo6/src/`, cible `axi4lite_io_linux` ;
- copie du binaire sur la carte avec `scp axi4lite_io_linux root@192.168.0.2:/root/`, puis exécution en `root` sur la carte.

Les tests côté Linux se sont déroulés correctement : lecture des switchs et touches sur le bridge, lecture de la constante AXI à `0xFF200000` avec `devmem`, et comportement attendu du programme sur la console série (boucle bridge puis test des registres et interaction avec l'IP lorsque le FPGA contient le bon design).

10 Reprogrammation du FPGA et problème de licence

Pour valider complètement le laboratoire 6, il faut reprogrammer le FPGA avec le bitstream du laboratoire 5 (`pgm_fpga.py` ou chargement du preloader / bitstream via les scripts du dépôt). Depuis la VM de laboratoire (`reds@vmeda2026`, dossier `de1_scripts`), l'exécution de `python3 upld_hps.py` a échoué.

Les messages indiquent une erreur JTAG / AJI (`AJI failed to send data`, `Encounter JTAG AJI error while accessing HPS`, `Fail to power up the System and Debug power`) et le lancement de `quartus_hps` se termine sans succès. Le journal mélange aussi du texte lié à la licence Intel (*Quartus Prime License Agreement*, etc.).

J'étais connecté au VPN et j'arrivais à ouvrir Quartus et Arm DS. En revanche, le *Programmer* en ligne de commande (`quartus_hps`, scripts du labo) a échoué avec des messages qui évoquent autant la licence Intel que des erreurs JTAG / AJI. J'ai l'impression que c'est un blocage lié à la licence. Comme la partie Linux sur la carte a fonctionné et que la programmation FPGA avait déjà marché au laboratoire précédent, je pars du principe que la liaison matérielle est correcte et que le problème vient surtout que je ne suis pas dans le réseau de la HEIG. Je ne pourrai vérifier et reprogrammer le `.rbf` du labo 5 de façon fiable qu'en étant de nouveau physiquement à la HEIG ; d'ici là, le design chargé au boot par U-Boot est resté celui par défaut pour les parties qui en dépendent.

11 Utilisation d'outil IA

Pour ce rapport, j'ai utilisé un assistant IA pour m'aider à structurer les étapes et à reformuler certaines phrases. Les manipulations techniques, les tests et les validations ont été réalisés par moi.